

UNIDAD 1

Fundamentos de la Programación Orientada a Objetos

Tema:

Técnicas de programación



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

GUÍA DE ESTUDIO SEMANA 2

ÍNDICE

1.1. INTRODUCCIÓN.....	1
1.2. RESULTADO DE APRENDIZAJE	2
1.3. RUTA DE APRENDIZAJE	2
1.4. DESARROLLO DE CONTENIDOS.....	3
1.4.1 Técnicas de programación en el contexto de la POO.....	3
1.4.2 Clases y objetos como base de la POO	3
1.4.3 Abstracción: representar lo esencial.....	4
1.4.4 Encapsulación: proteger la información.....	6
1.4.5 Herencia: reutilizar características	7
1.4.6 Polimorfismo: diferentes comportamientos.....	9
1.4.7 Relación entre las técnicas de la POO	10
1.4.8 Relación entre las técnicas de la POO	11
1.5. CONCLUSIONES.....	12
1.6. BIBLIOGRAFÍA.....	12

1.1. INTRODUCCIÓN

Las técnicas de programación constituyen un conjunto de principios y mecanismos que permiten organizar el desarrollo de software de manera más clara, eficiente y mantenible. En el contexto de la Programación Orientada a Objetos (POO), estas técnicas adquieren especial importancia porque permiten representar problemas reales mediante estructuras de código basadas en clases, objetos, atributos y métodos.

A medida que los programas aumentan en tamaño y complejidad, resulta más difícil modificar, ampliar y mantener el código si no existe una estructura adecuada. Por esta razón, la POO propone organizar las soluciones computacionales a partir de objetos que integran datos y comportamientos relacionados. Este enfoque permite construir aplicaciones más organizadas, reutilizables, escalables y fáciles de mantener.

La presente guía desarrolla las cuatro técnicas fundamentales de la Programación Orientada a Objetos: abstracción, encapsulación, herencia y polimorfismo. Estas técnicas permiten comprender cómo se diseña una solución orientada a objetos antes de escribir código, evitando estructuras desordenadas y favoreciendo un desarrollo más profesional.

Desde una perspectiva didáctica, las técnicas de la POO pueden comprenderse mediante ejemplos cotidianos. Un automóvil, una cuenta bancaria, una jerarquía de animales o una acción común como `hacer_sonido()` permiten visualizar cómo los objetos del mundo real pueden convertirse en modelos computacionales. Posteriormente, estos modelos pueden implementarse en lenguajes como Python, el cual facilita la creación de clases y objetos mediante una sintaxis clara y accesible (Downey, 2024).

El estudio de estas técnicas es fundamental para avanzar en la asignatura, ya que permitirá al estudiante comprender la lógica detrás del diseño de aplicaciones orientadas a objetos. Además, servirá como base para el desarrollo de ejercicios prácticos, talleres en

GitHub y actividades interactivas de refuerzo orientadas a consolidar los aprendizajes de la semana.

1.2. RESULTADO DE APRENDIZAJE

Interpreta los conceptos fundamentales de la Programación Orientada a Objetos para el desarrollo de soluciones computacionales de complejidad media.

1.3. RUTA DE APRENDIZAJE

La evaluación del aprendizaje se desarrollará mediante actividades correspondientes a los componentes en contacto con el docente, práctico-experimental y autónomo. Estas actividades están orientadas a fortalecer la comprensión conceptual y la aplicación básica de las técnicas fundamentales de la Programación Orientada a Objetos.

COMPONENTE	ACTIVIDAD	CALIFICACIÓN
Componente en contacto con el docente	Cuestionario sobre técnicas de programación Orientada a Objetos.	10 puntos
Componente práctico–experimental	Taller práctico de modelado básico de clases y objetos en Python con entrega mediante repositorio GitHub.	10 puntos
Componente autónomo	Actividad interactiva H5P de refuerzo sobre técnicas fundamentales de la Programación Orientada a Objetos.	10 puntos

Las actividades propuestas permitirán al estudiante identificar los fundamentos conceptuales de la POO, relacionarlos con situaciones cotidianas y aplicarlos progresivamente en ejercicios de programación desarrollados en Python.

1.4. DESARROLLO DE CONTENIDOS

1.4.1 Técnicas de programación en el contexto de la POO

Las técnicas de programación son procedimientos, principios o formas de organizar el código que permiten resolver problemas de manera estructurada. En la Programación Orientada a Objetos, estas técnicas se centran en modelar entidades mediante objetos que poseen datos y comportamientos.

La POO no se limita a escribir clases en un lenguaje de programación. Su propósito principal es construir modelos que representen adecuadamente un problema. Para lograrlo, se utilizan cuatro técnicas fundamentales: abstracción, encapsulación, herencia y polimorfismo. Cada una cumple una función específica en el diseño del software y, cuando se aplican correctamente, permiten crear sistemas más coherentes, flexibles y mantenibles (Hernández Bejarano & Baquero Rey, 2023).

En la diapositiva de introducción de la Semana 2 se muestra la transición de un sistema caótico hacia un sistema organizado. Esta idea representa uno de los principales aportes de la POO: transformar un conjunto desordenado de instrucciones en módulos relacionados, comprensibles y fáciles de modificar.

TÉCNICA	PROPÓSITO PRINCIPAL
Abstracción	Representar lo esencial de un objeto.
Encapsulación	Proteger los datos y controlar el acceso.
Herencia	Reutilizar atributos y métodos de clases existentes.
Polimorfismo	Permitir que una misma acción tenga distintos comportamientos.

1.4.2 Clases y objetos como base de la POO

Antes de estudiar cada técnica, es necesario comprender la relación entre clase y objeto. Una clase puede entenderse como un modelo, plantilla o estructura que define qué datos tendrá un objeto y qué acciones podrá realizar. Un objeto, en cambio, es una instancia concreta creada a partir de una clase.

Por ejemplo, la clase Automóvil puede definir atributos como marca, modelo y velocidad, así como métodos como acelerar() o frenar(). A partir de esa clase se pueden crear objetos concretos, como un automóvil Toyota, un automóvil Honda o un automóvil Ford. Todos comparten la misma estructura general, pero cada uno puede tener valores distintos en sus atributos.

Esta relación es fundamental porque las técnicas de abstracción, encapsulación, herencia y polimorfismo se implementan mediante clases y objetos. Por ello, comprender esta base permite interpretar con mayor claridad el diseño orientado a objetos y su aplicación en Python (Hernández Bejarano & Baquero Rey, 2025).

A continuación, se presenta un ejemplo sencillo en Python que muestra la definición de una clase y la creación de un objeto a partir de ella.

```
class Automovil:
    def __init__(self, marca, modelo, velocidad=0):
        self.marca = marca
        self.modelo = modelo
        self.velocidad = velocidad

    def acelerar(self):
        self.velocidad += 10

mi_auto = Automovil("Toyota", "Corolla")
mi_auto.acelerar()
```

1.4.3 Abstracción: representar lo esencial

La abstracción consiste en identificar las características y comportamientos más importantes de un objeto, ignorando los detalles que no son necesarios para resolver un problema. Esta técnica permite simplificar la realidad y convertirla en un modelo comprensible dentro de un programa.

En el ejemplo del automóvil, no es necesario representar cada tornillo interno, cada cable del motor o cada detalle mecánico si el problema únicamente requiere trabajar con la marca, el modelo y la velocidad del vehículo. La abstracción permite seleccionar aquello que resulta relevante para el sistema y descartar los detalles que no aportan a la solución.

Desde el punto de vista del diseño de software, la abstracción ayuda a reducir la complejidad. Un sistema puede estar compuesto por muchos elementos, pero el programador debe identificar cuáles son esenciales para cumplir el objetivo del programa. Esto evita construir modelos excesivamente detallados o difíciles de mantener.

En Python, la abstracción se evidencia al definir clases con atributos y métodos que representan únicamente los aspectos necesarios del objeto. La clase no busca copiar toda la realidad, sino construir una representación útil para resolver un problema específico (Boucheny, 2025).

A continuación, se presenta un ejemplo en Python que muestra cómo una clase puede representar únicamente las características esenciales de un automóvil, ignorando detalles que no son relevantes para el problema planteado.

```
class Auto:
    def __init__(self, marca, modelo, velocidad=0):
        self.marca = marca
        self.modelo = modelo
        self.velocidad = velocidad

    def acelerar(self):
        self.velocidad += 10

# El modelo conserva solo la información necesaria.
auto = Auto("Toyota", "Yaris")
auto.acelerar()
```

Idea clave: la abstracción permite enfocarse en lo que se necesita conocer del objeto, no en todos sus detalles internos.

1.4.4 Encapsulación: proteger la información

La encapsulación consiste en proteger los datos internos de un objeto y permitir su acceso únicamente mediante acciones controladas. En lugar de modificar directamente los atributos, se utilizan métodos que regulan cómo se consultan o cambian los datos.

El ejemplo de una cuenta bancaria permite comprender esta técnica. Un usuario puede depositar o retirar dinero, pero no debería modificar directamente el saldo sin ningún control. Si el saldo pudiera alterarse libremente, el sistema sería inseguro e inconsistente. Por ello, la encapsulación protege los datos y define operaciones válidas para interactuar con ellos.

En el desarrollo de software, la encapsulación mejora la seguridad lógica, la integridad de los datos y el mantenimiento del código. Si posteriormente se modifica la forma interna en que se calcula o almacena el saldo, el resto del sistema no necesariamente se verá afectado, siempre que los métodos públicos mantengan su funcionamiento.

En Python, la encapsulación se implementa mediante convenciones de acceso y métodos de control. Aunque Python no restringe el acceso de la misma forma que otros lenguajes, el uso de guion bajo en atributos como `_saldo` indica que se trata de un dato interno que no debería modificarse directamente desde fuera de la clase (Downey, 2024).

A continuación, se presenta un ejemplo en Python que muestra cómo los datos internos de un objeto pueden protegerse mediante métodos que controlan su acceso y modificación.

```
class CuentaBancaria:
    def __init__(self, saldo=0):
        self._saldo = saldo

    def depositar(self, monto):
```



```

        if monto > 0:
            self._saldo += monto

    def retirar(self, monto):
        if 0 < monto <= self._saldo:
            self._saldo -= monto

    def obtener_saldo(self):
        return self._saldo

cuenta = CuentaBancaria(100)
cuenta.depositar(50)
print(cuenta.obtener_saldo())

```

Idea clave: los datos se protegen para evitar cambios incorrectos, inseguros o no autorizados.

1.4.5 Herencia: reutilizar características

La herencia permite crear nuevas clases a partir de otras clases existentes, reutilizando atributos y comportamientos comunes. Esta técnica facilita la organización jerárquica del código y evita repetir instrucciones que ya han sido definidas en una clase general.

En el ejemplo de animales, la clase Animal puede representar características generales como nombre, edad o la capacidad de moverse. A partir de ella pueden crearse clases más específicas, como Perro y Gato, que heredan esas características comunes y agregan comportamientos propios, como ladrar o maullar.

La herencia favorece la reutilización del código, la consistencia estructural y la construcción progresiva de soluciones. Sin embargo, debe utilizarse con criterio. No toda relación entre clases implica herencia; esta técnica resulta adecuada cuando existe una relación del tipo “es un”. Por ejemplo, un perro es un animal y un gato es un animal.

En Python, una clase hija puede heredar de una clase padre colocando el nombre de la clase base entre paréntesis. Además, puede reutilizar el constructor de la clase padre mediante `super()`, lo cual permite inicializar los atributos comunes sin duplicar código (Hernández Bejarano & Baquero Rey, 2025).

A continuación, se presenta un ejemplo en Python que muestra cómo las clases Perro y Gato heredan atributos y comportamientos de la clase Animal.

```
class Animal:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def moverse(self):
        return f"{self.nombre} se está moviendo"

class Perro(Animal):
    def ladrar(self):
        return "Guau"

class Gato(Animal):
    def maullar(self):
        return "Miau"

perro = Perro("Firulais", 3)
gato = Gato("Michi", 2)
print(perro.moverse())
print(gato.moverse())
```

Idea clave: las clases hijas heredan características de una clase más general y pueden agregar sus propias funciones.

1.4.6 Polimorfismo: diferentes comportamientos

El polimorfismo permite que distintos objetos respondan de manera diferente a una misma acción. Esta técnica aporta flexibilidad al diseño del software, ya que permite utilizar una misma operación sobre objetos de diferentes clases, obteniendo resultados específicos según el tipo de objeto.

En el ejemplo de los animales, tanto un perro como un gato pueden ejecutar la acción `hacer_sonido()`. Sin embargo, el resultado no es el mismo: el perro ladra y el gato maúlla. La acción tiene el mismo nombre, pero el comportamiento cambia según el objeto que la ejecuta.

El polimorfismo permite escribir código más general y adaptable. En lugar de crear una función diferente para cada tipo de objeto, se puede invocar un mismo método y dejar que cada clase defina su propia respuesta. Esto reduce la dependencia entre partes del sistema y facilita la ampliación futura del programa.

En Python, el polimorfismo puede aplicarse mediante métodos con el mismo nombre en diferentes clases. También puede combinarse con herencia cuando una clase hija redefine un método heredado de la clase padre (Boucheny, 2025).

A continuación, se presenta un ejemplo en Python que muestra cómo diferentes clases pueden responder de manera distinta a una misma acción mediante el uso de métodos con el mismo nombre.

```
class Animal:
    def hacer_sonido(self):
        pass

class Perro(Animal):
    def hacer_sonido(self):
        return "Guau"
```

```
class Gato(Animal):  
    def hacer_sonido(self):  
        return "Miau"
```

```
animales = [Perro(), Gato()]
```

```
for animal in animales:  
    print(animal.hacer_sonido())
```

Idea clave: una misma acción puede producir resultados diferentes según el objeto que la ejecute.

1.4.7 Relación entre las técnicas de la POO

Las cuatro técnicas de la Programación Orientada a Objetos no deben estudiarse como conceptos aislados. En una aplicación real, suelen trabajar de manera conjunta para permitir un diseño de software más sólido.

La abstracción permite decidir qué información debe representarse. La encapsulación protege esa información y regula su acceso. La herencia permite reutilizar características comunes entre clases relacionadas. Finalmente, el polimorfismo permite que diferentes objetos respondan de manera particular ante una misma operación.

Por ejemplo, en un sistema de gestión veterinaria se podría crear una clase `Animal` que contenga atributos generales como nombre, edad y especie. Luego, clases como `Perro`, `Gato` o `Ave` podrían heredar de `Animal`. Cada clase podría redefinir el método `hacer_sonido()` para expresar un comportamiento diferente. Al mismo tiempo, los datos de cada animal podrían mantenerse encapsulados mediante métodos de acceso y modificación controlados.

Esta integración demuestra que la POO no se basa únicamente en escribir código, sino en diseñar una estructura lógica que represente adecuadamente el problema. Por ello, el

dominio de estas técnicas resulta esencial para construir aplicaciones comprensibles, extensibles y mantenibles (Hernández Bejarano & Baquero Rey, 2023).

TÉCNICA	PREGUNTA GUÍA	RESULTADO EN EL DISEÑO
Abstracción	¿Qué datos y acciones son realmente importantes?	Modelo simple y enfocado.
Encapsulación	¿Cómo protejo los datos internos?	Mayor seguridad e integridad.
Herencia	¿Qué características pueden reutilizarse?	Menos duplicación de código.
Polimorfismo	¿Cómo puede variar una misma acción?	Mayor flexibilidad y extensibilidad.

1.4.8 Relación entre las técnicas de la POO

Python es un lenguaje adecuado para iniciar el estudio de la Programación Orientada a Objetos debido a su sintaxis clara y su capacidad para representar conceptos complejos mediante estructuras sencillas. El lenguaje permite crear clases, instanciar objetos, definir métodos, aplicar herencia y aprovechar el polimorfismo sin requerir una sintaxis excesivamente compleja.

En la Semana 2, el propósito no es profundizar todavía en todos los detalles técnicos de la implementación, sino comprender la función de cada técnica dentro del diseño del software. Posteriormente, estos conceptos se aplicarán en laboratorios, talleres y ejercicios prácticos orientados a construir programas más organizados.

La siguiente tabla resume cómo se aplican las técnicas fundamentales de la POO en Python:

TÉCNICA	APLICACIÓN	CÓMO SE USA EN PYTHON
Abstracción	Clases	Se crea una clase que representa la esencia de un objeto, seleccionando los datos y métodos necesarios.
Encapsulación	Atributos y métodos	Se protegen los datos internos y se controla el acceso mediante métodos.
Herencia	Clases derivadas	Se crean nuevas clases que heredan atributos y métodos de clases existentes.
Polimorfismo	Métodos redefinidos	Distintas clases pueden implementar un mismo método de diferentes formas.

1.5. CONCLUSIONES

Las técnicas fundamentales de la Programación Orientada a Objetos permiten organizar el software mediante modelos más claros, reutilizables y fáciles de mantener.

La abstracción permite representar solo los aspectos esenciales de un objeto, reduciendo la complejidad del problema y facilitando su implementación.

La encapsulación protege los datos internos de los objetos y permite controlar el acceso a la información mediante métodos definidos dentro de la clase.

La herencia favorece la reutilización del código al permitir que nuevas clases aprovechen atributos y comportamientos de clases existentes.

El polimorfismo permite que distintos objetos respondan de manera diferente a una misma acción, aportando flexibilidad y extensibilidad al desarrollo de software.

Python facilita la aplicación de estas técnicas mediante una sintaxis clara, lo que lo convierte en una herramienta adecuada para iniciar el aprendizaje de la Programación Orientada a Objetos.

1.6. BIBLIOGRAFÍA

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.). Ediciones ENI.

Downey, A. (2024). Think Python (3rd ed.). O'Reilly Media.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.